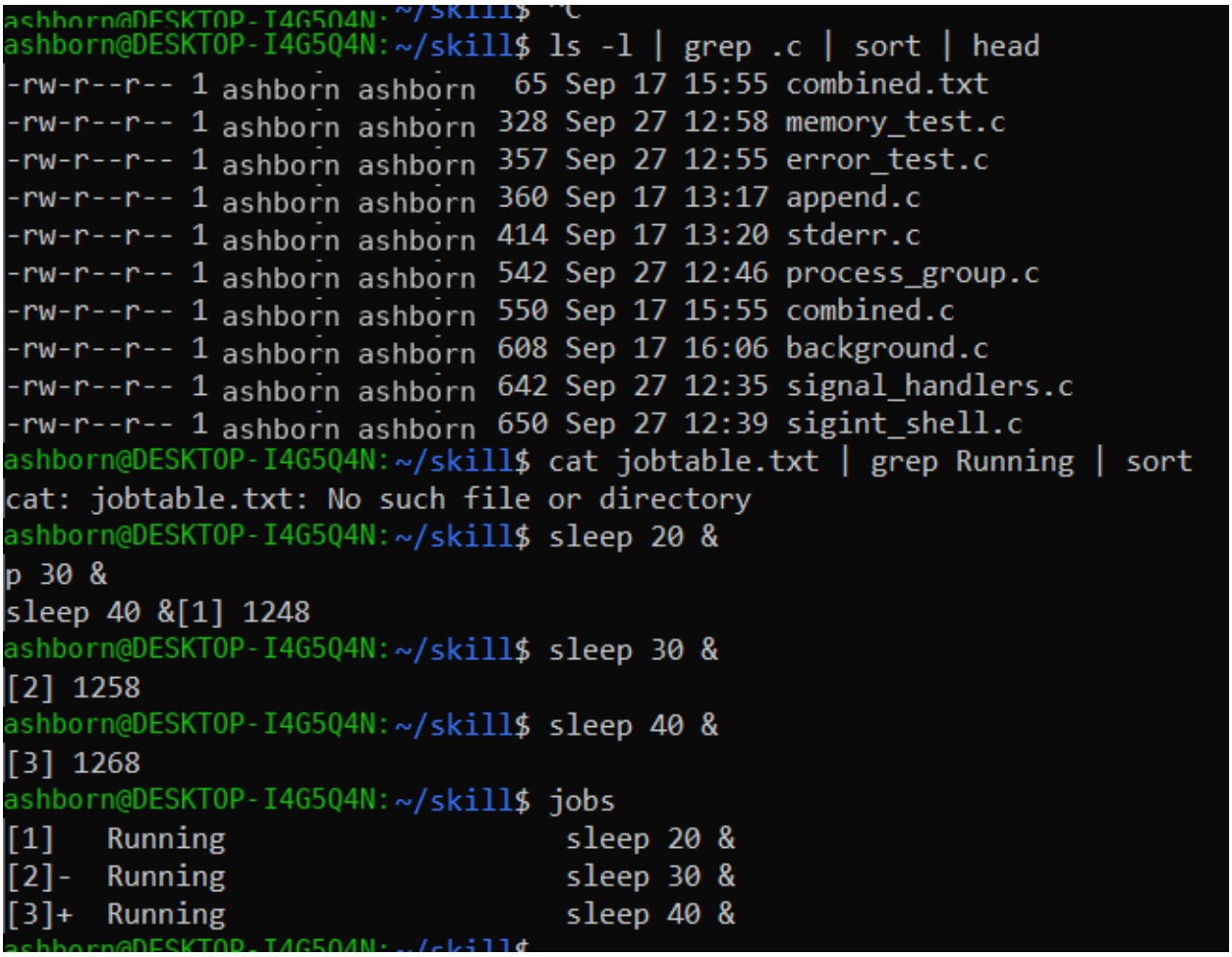
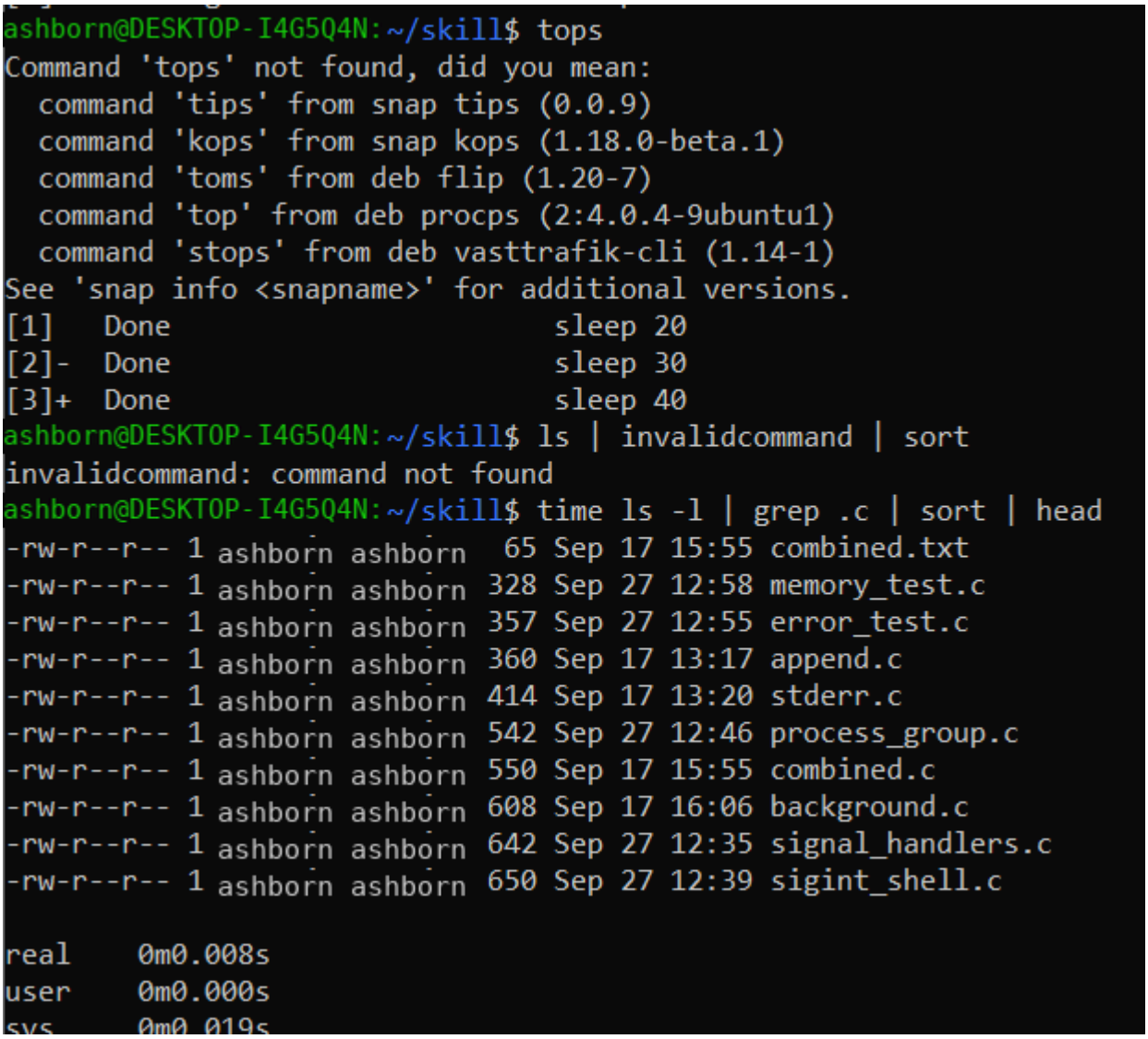
WEEK-12

Task-1: To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance

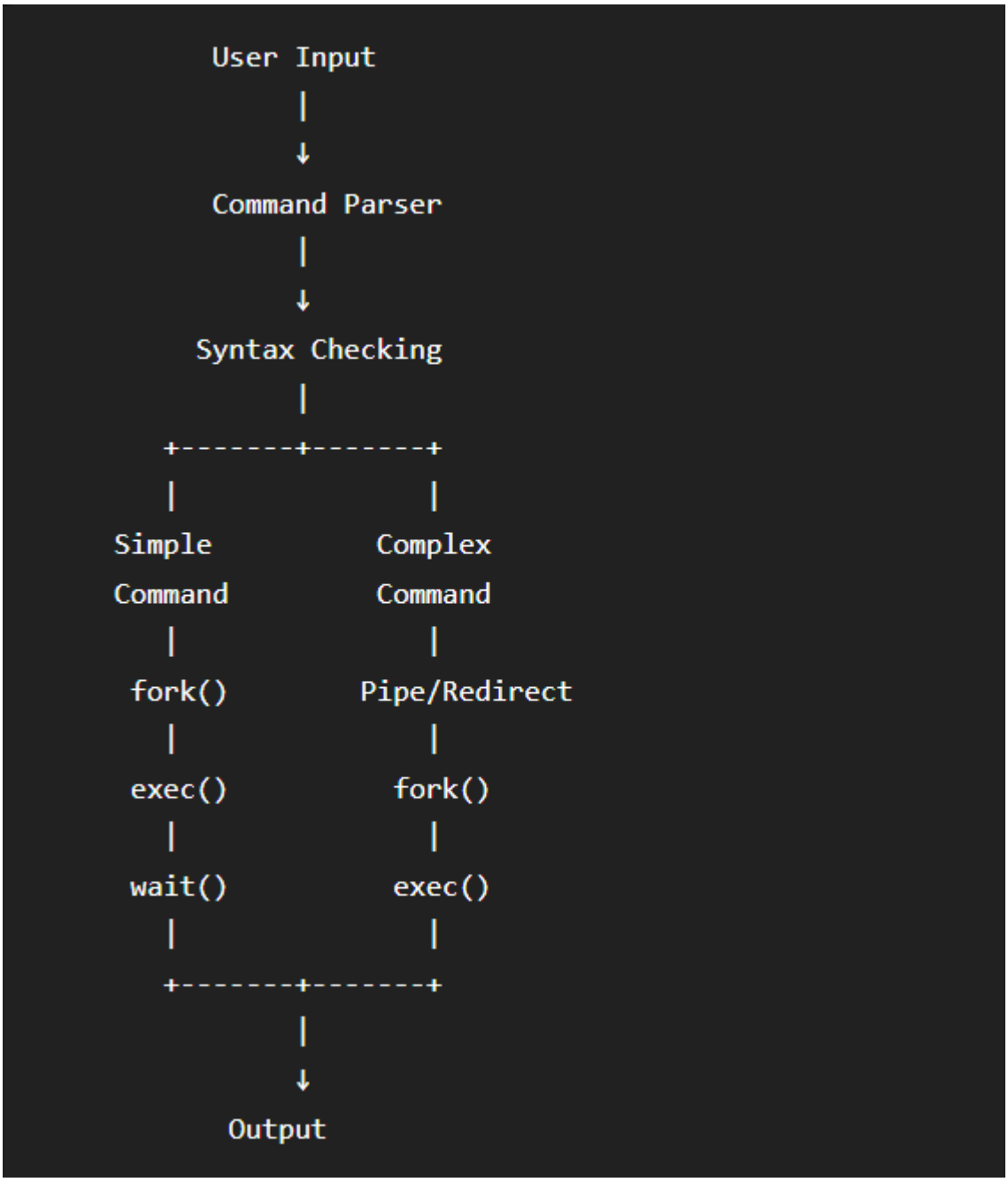




Report: Large pipelines and multiple background jobs were executed to test the stability of the system.
CPU and memory usage were monitored using Linux process-monitoring commands.
Failed commands were also tested to check whether errors were handled without crashing the program.
Execution time was measured to study the performance of pipelines and process management.

Task-2: To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation

Architecture:



Important system calls

System Call	Purpose
fork()	Creates a child process
exec()	Executes a command
wait()/waitpid()	Waits for child processes
pipe()	Creates communication channel
dup2()	Performs I/O redirection
open()	Opens files
read()	Reads data
write()	Writes data
close()	Closes file descriptors
kill()	Sends signals

Report:

The architecture and execution flow of the project were documented using diagrams and descriptions.
The major system calls used for process creation, pipes, redirection, and signals were explained.
Important design decisions and the interaction between different modules were recorded.
The documentation was reviewed to provide a clear understanding of the complete project.

Task-3: To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

```
  GNU nano 8.7.1                                README.md *
# Linux Shell Project

## Description
A Linux shell project demonstrating process management,
pipes, I/O redirection, signals, job control and
inter-process communication.

## Features

- Command execution
- Pipes
- Input/output redirection
- Background jobs
- Job control
- Signal handling
- Process groups
- Error handling
- Memory testing

## Compilation

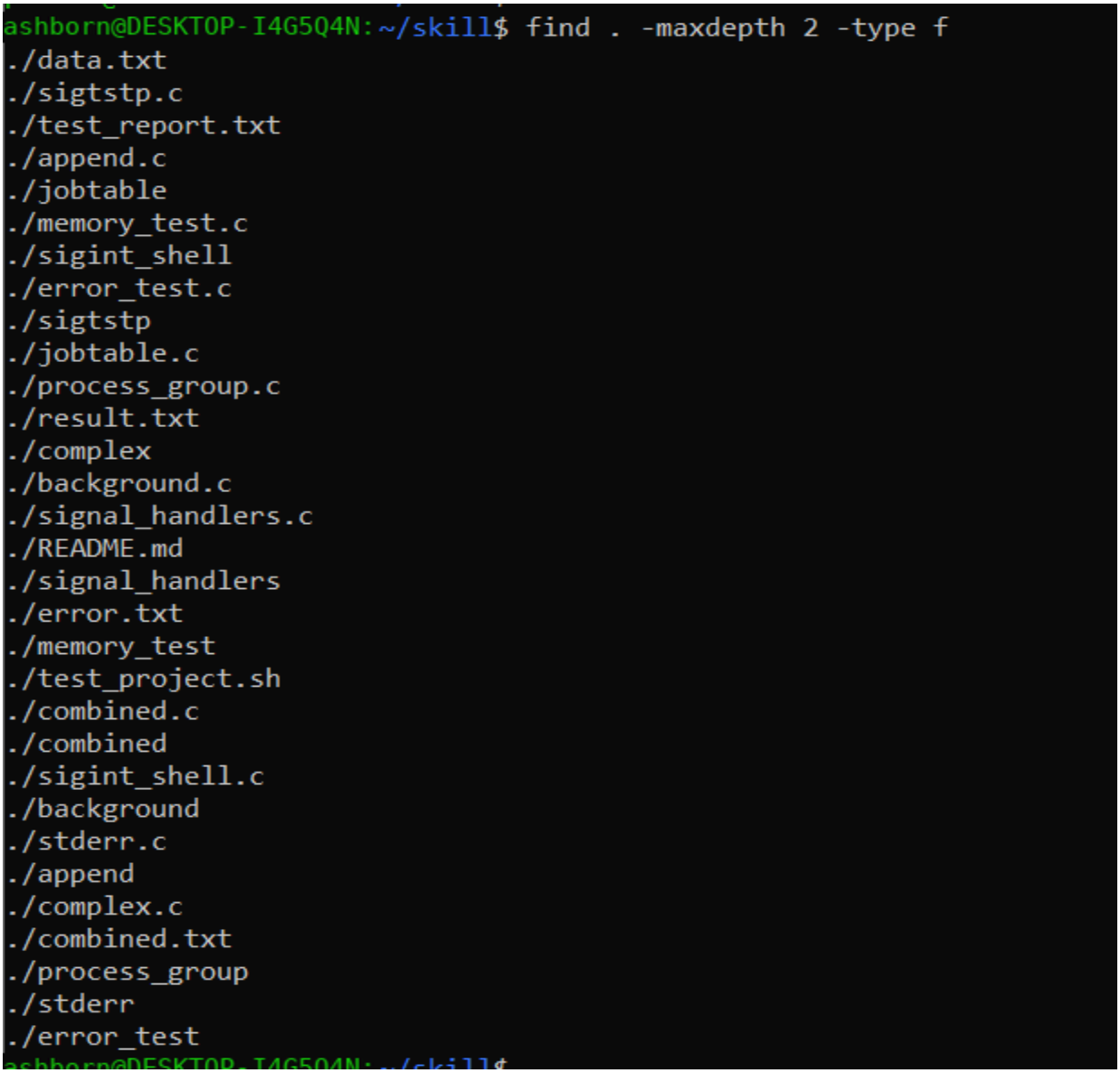
gcc program.c -o program

## Execution

./program

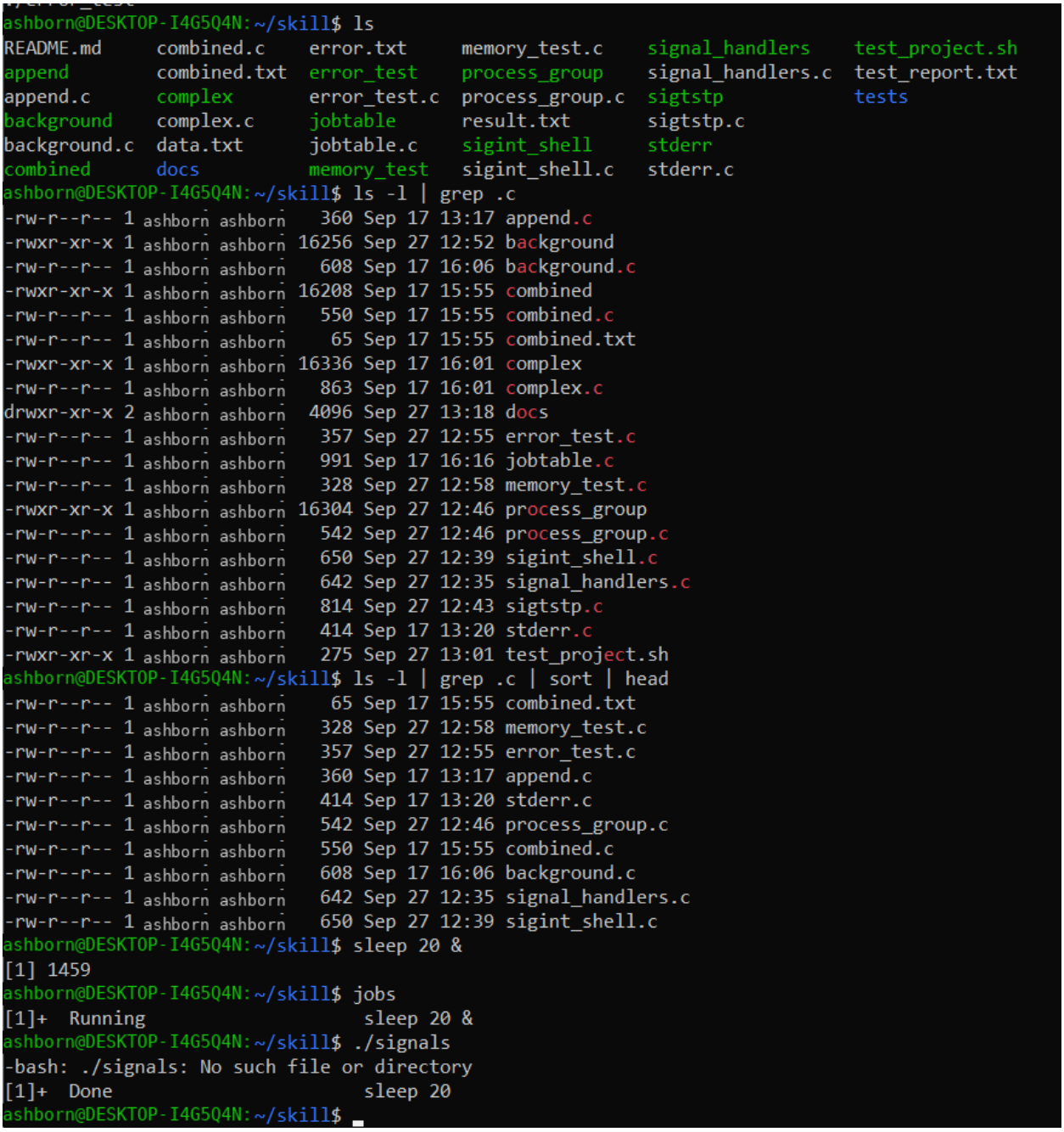
## System Calls

fork()
exec()
pipe()
dup2()
waitpid()
open()
read()
write()
close()
kill()
```



Report: The project source code and supporting files were organized into a clear repository structure.
A README file was added to explain the project, features, compilation, and execution steps.
The repository was checked to ensure that the required source files and documentation were present.
The final deliverables were reviewed and a version tag was created for the completed project.

Task-4: TO Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review



Report: Final demonstration scenarios were prepared to verify the major features of the project. Pipelines, background jobs, signals, and process groups were demonstrated during the review. The complete project was checked for functionality, documentation, source organization, and reliability.

The final testing confirmed that the implemented components were ready for presentation and submission.